Computer Networks 57 (2013) 243–257

Contents lists available at SciVerse ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

VASE: Filtering IP spoofing traffic with agility

Guang Yao *, Jun Bi, Peiyao Xiao

Tsinghua University, FIT Building Room 4-204, Beijing 100084, China

ARTICLE INFO

Article history:
Received 8 November 2011
Received in revised form 24 May 2012
Accepted 9 August 2012
Available online 21 September 2012

Keywords:
TCP/IP
IP spoofing
Agility

ABSTRACT

Filtering out traffic with forged source address on routers can significantly improve the security of Internet. However, despite intermittent IP spoofing attacks, existing filtering mechanisms inspect each packet all the time, consuming considerable resource on routers even there is no spoofing at all. This article considers the requirement for a solution performing IP spoofing filtering with agility, which consumes resource in proportional to the size of attack. A novel IP spoofing filtering mechanism named Virtual Anti-Spoofing Edge (VASE) is proposed in this article. VASE uses sampling and on-demand filter configuration to reduce unnecessary overhead in peace time. The evaluation based on simulation shows VASE has obvious advantages over commonly used mechanisms in various scenarios. VASE is fully compatible with current IP spoofing filtering practices and can be implemented with commodity routers. In the campus network of Tsinghua University, VASE is providing real benefits.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

IP spoofing [1], which means attackers make use of forged source IP addresses to launch attacks, has been long recognized as a serious security problem on the Internet. By modifying the source address of attacking traffic to address assigned to others or even unassigned, an attacker can hide its actual location, or circumvent access control rules on source, or utilize the flaw of the victim system, or gain the capability of launching reflection based attack. A number of notorious attacks rely on IP spoofing, including SYN flooding [2], Smurf [3], DNS amplification [4], etc. In 2006, a DNS amplification attack against Top Level Domain (TLD) name server severely degraded the capacity of the TLD name server for several days [5]. According to the observation of CAIDA, there are at least 3000 (D)DoS attacks based on IP spoofing in a week [6]. In the NANOG 50th meeting, a presentation from ARBOR shows spoofing is quite significant in despite of large size of zombie networks [7].

To filter IP spoofing traffic, Internet Service Providers (ISPs) generally deploy filtering mechanisms on routers to perform prefix granularity filtering, i.e., Ingress/Egress Filtering [8]. The measurement from MIT Spoofer project [9] shows Ingress Filtering (or similar mechanisms) has been deployed on 76.2% ASs of all the participants. The statistic work of ARBOR Networks shows Ingress Filtering has been deployed on approaching 60% ISP networks investigated [10]. uRPF [11] is the most common approach to implement Ingress Filtering. It filters the packet based on looking up in the forwarding table to check whether the incoming interface of a packet matches the forwarding interface to the source. However, due to the requirement of robustness and load-balance, the Internet network path is becoming more and more diversified and asymmetric. uRPF will drop legitimate traffic whenever the incoming interface is not exactly the forwarding interface resulting from asymmetric routing. In the real deployment, uRPF is only used on the access port of the edge network. Such deployment does not have defense capability against spoofing traffic from undeployed networks or other domains. The other way to implement Ingress Filtering is using Access Control List (ACL). Whenever ACL is

* Corresponding author. Tel./fax: +86 010 62795818x6948.
E-mail address: yaoguang.china@gmail.com (G. Yao).

1389-1286/$ - see front matter © 2012 Elsevier B.V. All rights reserved.
http://dx.doi.org/10.1016/j.comnet.2012.08.018

processed by the processor of the router, the router must scan the ACL entries until finding a match. It is known such ACL will degrade the performance of the router greatly, especially whenever the network load is heavy [12]. Though packet inspection with TCAM (Ternary Content Addressable Memory) based ACL can be performed at line speed, TCAM is expensive and limited resource on routers, shared by routing, QoS and other functions requiring IP looking-up. Unfortunately, the explosion of route entries has made a large number of routers just load all the route entries [13]. The required TCAM resource in a spoofing filtering mechanism is determined by the filtering granularity. Prefix granularity IP spoofing filtering on routers will cost at least the same amount TCAM entries of the routing function, because it must map each prefix to the corresponding incoming interface, just as the routing function must map each prefix to the corresponding forwarding interface.

There are also other proposed spoofing filtering mechanisms, which are generally classified into filtering-on-path [14–17] and end-to-end authentication [18,19]. Till now no mechanism is known to obtain better data plane processing performance and less resource consumption than TCAM based ACL. In these mechanisms, other than the data plane cost, control plane also generates unnecessary cost when no spoofing happens. Take route based mechanism as an example, whenever route changes, the filtering rules on routers must be reconstructed just to avoid discarding legitimate packet even there is no spoofing at all [15].

Despite the processing overhead on all packets and permanent occupied resource, there is not always spoofing based attack in the network. IP spoofing is typically used to launch DDoS attack, which is generally short and fierce. According to the analysis of CAIDA [6], 80% attacks are less than 30 min, but only 1% attacks are more than 10 h. In a long period, the ratio of spoofing traffic is trivial to legitimate traffic; however, to filter out the spoofing traffic, the load of the router is significantly increased all the time.

Though serious resource waste exists in current anti-spoofing mechanisms, there has been no solution known to reduce the waste. In authors' opinion, the key factor to reduce the waste is to perform filtering with agility. In this article, a novel spoofing IP flow filtering mechanism named Virtual Anti-Spoofing Edge (VASE) is proposed to provide agile IP spoofing filtering capability. It minimizes the data plane cost when no spoofing happens and provides maximum filtering capability whenever spoofing is detected. VASE forms a protective perimeter which protects inside nodes from spoofing from the outside. Routers on the perimeter perform sampling but do not check each packet. The samples are redirected to the VASE server which captures and analyses the samples. Instead of deploying always configured filters on routers, VASE adaptively configures filters on routers in attack and removes filters if no spoofing is detected. A novel filter generation mechanism is designed to help identify spoofing packet. Though simulation, we found VASE can greatly reduce the resource requirement on routers without reducing filtering ability. VASE is fully compatible with existing commercial routers, and it has been deployed in the campus network of Tsinghua University for almost a year.

This work is of following contributions:

- It is the first work known concerning the unnecessary cost of existing IP spoofing filtering mechanisms. There is not always spoofing traffic in the network. Even when there is a spoofing attack, it is usually of no need to configure all the filtering rules on routers. Always-and-all configured filtering rules introduce significant overhead on routers and may cause discarding legitimate traffic in case of route change. It proposes the object that spoofing filtering must be performed with agility.
- It proposes a solution, VASE, which enables IP spoofing filtering with agility. VASE configures filtering rules on the router on-demand to avoid unnecessary cost on the routers when there is no spoofing traffic. On-demand filtering rule generation mechanism is used to reduce the overhead of control plane. Through make use of the existing interfaces of current routers, VASE is fully compatible with current commercial routers and is easy to deploy in the real network.
- VASE has been deployed in real IPv6 networks for around one year. It has captured quite a large number of IP address misuses from a private 6to4 tunnel in reality. Excluding the misuses, over the last year, VASE server only configured one rule hundreds of times on one router to filter spoofing caused by a controversial configuration error. The deployment proves VASE is a feasible solution to avoid unnecessary cost in peace time.

The following of this article is structured as follows. The following section is the detailed description of the VASE architecture. Then we evaluate VASE through simulation and analyze the result of the simulation. An implementation of VASE and its real deployment is introduced in the following section. Then main IP spoofing prevention mechanisms are summarized. The interfaces of commodity routers used by VASE are introduced as background. In the next section, we discuss the deployment and security problems of VASE, and conclude the contributions with future work.

2. Virtual Anti-Spoofing Edge architecture

2.1. Overview

The architecture of VASE is illustrated in Fig. 1. To avoid performing validation on each packet, filtering is not always performed; instead, sampling is performed on routers. A controller named VASE server is used to collect sample packet from routers. To identify spoofing from the sampled packets, VASE server generates filtering rules based on flow path computation which is introduced in Section 2.3. If a sample is matched by filtering rule, adaptive rules will be configured on the corresponding interface. The configured rules will be removed if no spoofing is detected for a certain time later. In this way, the unnecessary cost in peace time can be reduced, and spoofing traffic can still be filtered effectively.

Such architecture is inspired by the recent emergences of researches which propose using a server to provide a

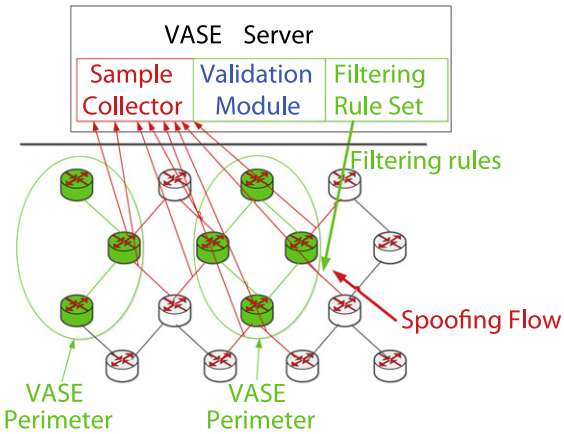


Fig. 1. The Virtual Anti-Spoofing Edge architecture.

global view of the network [20–22]. The global view can help the network perform load balance or other tasks better than when they are performed by each router with a local view. IP spoofing identification requires a lot of resource on routers; however, if this task is performed by a separate server, the resource on routers can be saved. Besides, the server can perform better filtering because it has the global view of the network.

Another feature to reduce data plane cost is the protection perimeter. For a network with a certain scale, the number of interfaces managed by the VASE server can be enormous. In order to make the solution scalable, a perimeter concept is introduced. The routers chosen to filter spoofing traffic can form one or more perimeters separating the inside routers from the other part of the network, as illustrated in Fig. 1. Inside the perimeters, sampling and filtering are only performed at the interfaces where the perimeter connects with the other part of the network and the interface attached by the stub network. There is no need to check packet on the interfaces connected to other routers inside the perimeter, because the incoming traffic has either been validated whenever it intersects the perimeter, or leaves the stub network. The perimeter concept is essential: however large the VASE coverage is, the number of interfaces to manage is only the sum of the amount of stub networks in the perimeter and the amount of links intersecting the perimeter. It is worth noting that whenever all the routers are contained in VASE, filtering will be only performed on interfaces attached by the stub network.

Hereafter, the perimeters are named by VASE perimeter and the routers inside the perimeters are named by VASE router. Interfaces on which sampling and filtering are performed are denoted by VASE interface, specially, the interfaces on links that intersect VASE perimeter are denoted by VASE border interface.

2.2. Traffic sampling and sample collecting

Whenever routers to perform filtering are determined, sampling is configured on the VASE interfaces. Traffics arriving at VASE interfaces are sampled to the VASE server by sampling mechanism on routers, discussed in Section 5.2.

The sampling rate must be set with the considering of the tradeoff between the recall ratio of spoofing flow and load on routers. To capture an attack, at least one spoofing packet must be sampled in the attacking period. Three parameters are essential for designing an appropriate sampling rate: (1) the duration of attacks; (2) the ratio of spoofing packets to all the packets; (3) the rate of packet on the link.

Inspired by the shape of pdf (probability density function) diagram of duration in [7], we use log-normal distribution $Log - N(\mu, \sigma^2)$ to model the spoofing flow duration, as illustrated in Fig. 2. Levenberg–Marquardt algorithm [23] is used to get the nonlinear least squares fitting result based on samples from the statistic result of [7], the estimated parameters based on these given values are $\mu = 3.0541$, $\sigma^2 = 1.0804$. Based on the model of attack duration, more than 99.9% spoofing flows last more than 0.85 min. Note that attacks with less duration than 1 min are eliminated from the statistic result; thus the result is biased. However, an attack of very short duration is not of great damage. Thus, the bias is not vital.

To launch an successful (D)DoS attack, attackers will try their best to utilize the available bandwidth to send attacking traffic. Denote the ratio of spoofing packet by r. If attacks are required to be captured with probability no less than P, the required sampling times c is:

$$c \geqslant \frac{\ln(1-P)}{\ln(1-r)}$$

The corresponding sampling rate is determined by the packet per second (PPS) on interface. Denote the PPS of link by pps. If requiring capturing attacks with duration no less than m seconds, the required sampling rate is:

$$rate \geqslant \frac{c}{pps \cdot m} \geqslant \frac{\ln(1-P)}{pps \cdot m \cdot \ln(1-r)}$$

Take 1% link occupation as a serious attack and 90% as the required capture rate. It requires $1{:}10^6$ sampling rate on a 10 GE interface when average packet size is 1500 - bytes, and the link is of 10% load, or $1{:}2.2 \times 10^7$ in the case that link is of 100% load and average packet size is 64 bytes.

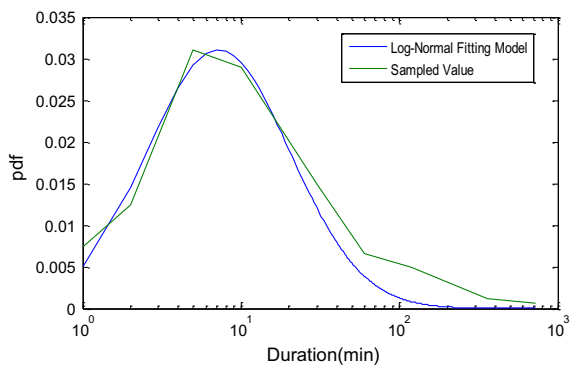


Fig. 2. The fitting model of duration distribution of spoofing attacks.

The cost of such sampling rates is trivial for current routers. For a network with 1000 10 GE VASE interfaces configured, the collection rate on the VASE server is less than 88.9 kpps, or approximately 100 Mbps. If using a 10 GE interface to collect the sampled packet, sampling can be configured on 10^5 10 GE interfaces, which is much larger than current network size.

If the higher sampling rate is set to capture spoofing flow with short duration, it is suggested to use multiple collectors. If the link occupancy by spoofing traffic is too trivial, it is difficult for the attack to vanquish the victim, or at least it is not of significant value to filter the spoofing traffic.

2.3. Check and confirm

A validation module on the VASE server performs check on samples according to bindings between router interfaces and prefixes/flows. However, there are no existing bindings on the VASE server; thus, the bindings must be built from another mechanism. There is no need to tightly couple VASE with a specified filter rule generation mechanism. VASE can cooperate with any filtering rule generation mechanism, e.g., SAVE [15], manual configuration, or even uRPF.

Considering the backwards compatibility requirements, only direction based filtering can be performed on routers through well-supported ACL function. Thus, the filtering rules must be route based, i.e., filtering rules are generated based on "predicting" the incoming interface of prefixes/flows. Considering the generated filter rule may violate instant flow path due to routing change, false positive and false negative can be introduced. False negative is generally not regarded as a serious problem. A number of spoofing traffic will be let pass, but this problem can be eliminated through performing routine synchronization. However, false positive can cause legitimate traffic to be filtered.

In uRPF, the route path between two nodes can be asymmetric due to the delay of routing convergence. Until the route convergences, uRPF will discard legitimate traffic. This outer-sync problem exists even if an active forwarding information distribution protocol is designed. For example, though SAVE can actively distribute the forwarding information, it still requires time to propagate the update. Thus, mechanisms using always configured filtering rules can hardly avoid the false positive caused by route change. However, most of the time the forwarding information distribution and regeneration of filtering rules are unnecessary, because there is no spoofing at all.

Considering the necessity to eliminate the false positive from outer-sync and reduce the control panel cost and, a passive rule generation mechanism is designed. In this mechanism, whenever required, filtering rules are generated through calculating the flow path. The output of the algorithm is the flows to deny on each VASE interface. However, the filtering rules are not actively synced with the routing state. Instead, the checking procedure confirms the consistency of the filtering rules and the instant flow path before announcing a spoofing traffic. Whenever a spoofing packet is found, instead of announcing spoofing immediately, the filtering rules are synced with the routing state. Only when a tetrad is still invalid after confirmation, a spoofing event is announced, and filter configuration is triggered. Though false negative can be increased due to route change, the false positive is avoided without significant cost.

The filtering rule generation mechanism for VASE is specified as follows. This mechanism makes use of the global view on the VASE server to generate more effective filtering rules than mechanisms based on local view of each router.

2.3.1. Flows with complete path known

We can calculate the whole path of a part of flows based on collected interface table and routing table from routers. Thus, for any router, it can be determined whether these flows can pass it, and through which interface(s) if the router is on the path of the flow. If a flow passes the router through an interface not on the calculated path, the flow must be with spoofing source. Only flow with forged source can travel along a path violating the calculated path for the flow.

Formally, for any flow (s,d) in the whole flow space F, it will either arrive at one or more interfaces of a router, or does not pass the router at all. The flows not passing a router are denoted by arriving on a virtual interface on the router, indexing by 0. Thus, if we get known the paths of all flows, we can build a map from interface to the flows that must be permitted on the interface. Denote the set of flows on the interface i of router r by F_{ri}. For a router with N interfaces, all the flow sets can be listed as $\{F_{r0}, F_{r1}, F_{r2}, F_{r3}, \ldots, F_{rN}\}$, in which $F_{ri} \subseteq F$, and $\bigcup_{i=0}^{N} F_{ri} = F$. F_{r0} contains the flows not passing the router.

However, to get known all the flow paths, we must collect MIB from all the routers in the network, which is generally difficult. In general, we can only get MIB from a part of routers, and compute complete paths of a part of the flows. Denote the set of flows known complete path by f. For any router, we can still map each flow in f to its interface: $\{f_{r0}, f_{r1}, f_{r2}, f_{r3}, \ldots, f_{rN}\}$, in which $f_{ri} \subseteq f$, $f_{ri} \subseteq F_{ri}$, and $\bigcup_{i=0}^{N} f_{ri} = f$. We denote the mapping from interfaces to the corresponding flows by *FlowTable*. As an instance, Fig. 3. illustrates the FlowTable on router E whenever collecting route table from router A–E.

If we find a packet (s,d) from interface i of router r, if $(s,d) \in f_{ri}$, the flow must be allowed because it follows the calculated data path. If $(s,d) \notin f_{ri}$, $(s,d) \in f_{rj}$, $j \neq i$, the flow violates calculated data path; thus it must be a spoofing flow. If $(s,d) \notin f$, the data path of the flow is unknown; thus it can arrive at any interface of the router.

In summary, we can perform *maximum direction based filtering* [14] on VASE interfaces, which can be represented as the following function:

$$V(r,i,s,d) = \begin{cases} true & (s,d) \in f_{ri} \\ false & (s,d) \notin f_{ri}, (s,d) \in f_{rj}, i \neq j \\ true & (s,d) \notin f \end{cases}$$

Obviously, the difference between the performance of VASE and a perfect direction based filtering mechanism

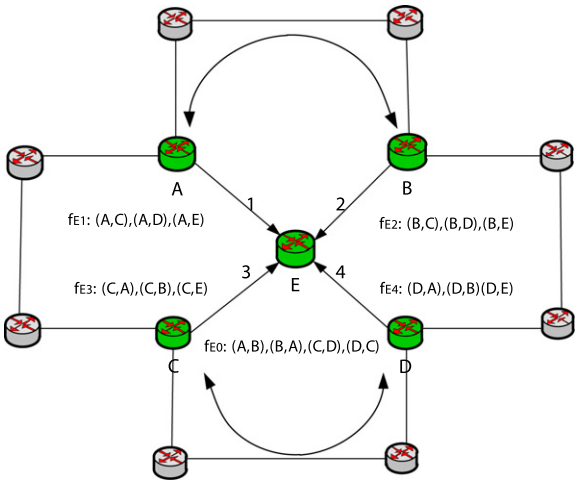


Fig. 3. Flow table generated on router E from collecting MIBs from routers A–E.

(which can find exact flow set for each interface) can be represented by the difference between f and F.

Now take the perimeter into consideration. If the routers to get MIB are all inside VASE, the interfaces through which flows originating from outside of VASE cannot be decided. Thus, a straightforward filtering mechanism can be made: filtering f on the VASE interfaces.

To make VASE have an advantage in filtering against Ingress Filtering, MIBs must be collected from at least two neighboring routers. Whenever MIBs can be collected from all the routers, perfect direction based filtering can be performed on routers selected to perform filtering.

2.3.2. Semi-path flows

Other than the flows whose complete path can be calculated, there are a number of flows whose paths from the origin to the VASE can be calculated, but the path is incomplete from collected information, as described in Fig. 4. The calculated path can also be used in filtering. When one such flow is calculated to arrive at one of the VASE interfaces on the VASE border router, if such flows are found arriving at another interface of the VASE router, the flow can be regarded as spoofing, because there should be no loop in the routing. However, there is no assurance whether such flows can arrive on other routers; thus such flow should be let pass whenever arriving at other routers.

Formally, for any flow (s,d) whose complete path cannot be calculated but the path from its origin to the VASE can be calculated, if it passes interface i on router r, we denote such flow by f'_{ri}. Denote the set containing all such flows by f', and such flows that pass router r by f'_r. The filtering function is changed to:

$$V(r,i,s,d) = \begin{cases} true & (s,d) \in f_{ri} \cup f'_{ri} \\ false & (s,d) \notin f_{ri} \cup f'_{ri}, (s,d) \in f_{rj} \cup f'_{rj}, i \neq j \\ true & (s,d) \in f', (s,d) \notin f \\ true & (s,d) \notin f \cup f' \end{cases}$$

2.4. Filter configuration

Whenever VASE server detects a spoofing sample, matching filters will be configured to cut the spoofing flow. To deny spoofing flows, corresponding ACL rules should be configured on routers. However, the rule number can be enormous. For instance, to set up a complete semi-maximum filtering [14] on one router, the number of rules is approximately the same as the number of route entries. If a router performs a complete maximum filtering, the rule number on the router is the number of flows that can pass it after aggregation. If configuring all the filters on one router, the performance of the router will be degraded seriously, and a lot of resources will be occupied.

Besides, the configuration of ACL rules will usually cost a lot of time. According to an experiment on a commodity router, the configuration time is in direct proportion to the number of rules, as illustrated in Fig. 5. Thus, if performing an all-in-one ACL rule configuration on routers, the high latency will make the filtering not timely. The situation will be even worse if the route changes during or shortly after configuration.

However, if only configuring the deny rule for the detected flow is usually meaningless in preventing spoofing

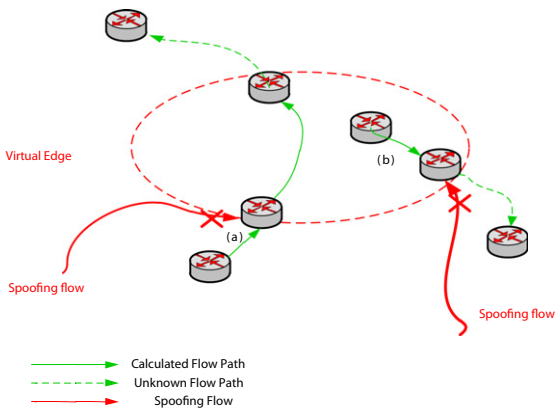


Fig. 4. Flows with incomplete path known: (a) flow originated from the outside of VASE; (b) flow originated from the inside of VASE.

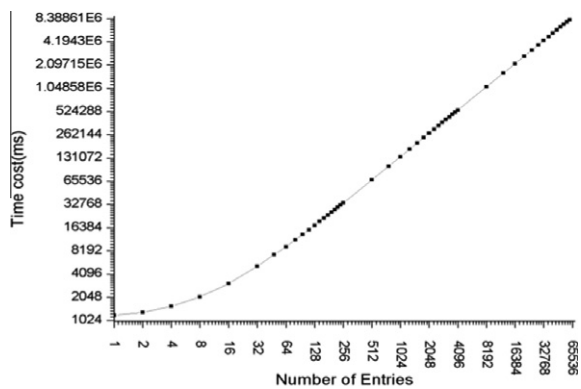


Fig. 5. Time cost in filtering rule configuration.

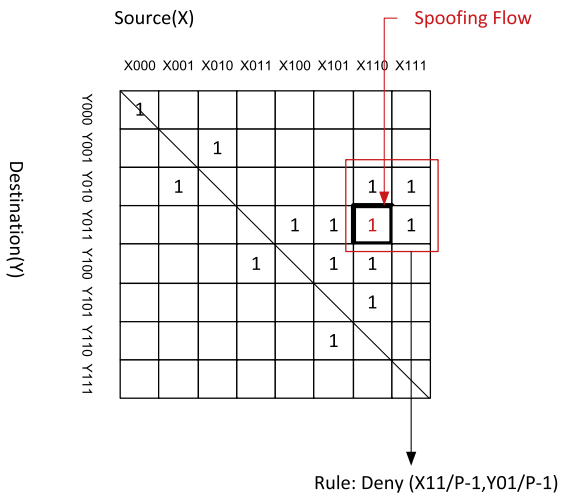


Fig. 6. Filter bitmap and rule generation.

Table 1
Algorithm for searching maximum rectangle.

```
1. PrefixPair (SourcePrefix x, DestPrefix y, SourcePrefix Length i,
   DestPrefixLength j)
2.   If (no IsAllOneSpace (x,y,i,j))
3.     Return MaximumValue;
4.   Else
5.     Return Min (i + j, PrefixPair (x,y,i − 1,j), PrefixPair
   (x,y,i,j − 1))
```

based attack, because generally attackers use variable address space as source or destination. To tradeoff between agility and effectiveness, we designed an adaptive filters configuration scheme.

Known spoofing based attack is with a specified target (DDoS) or source (reflection based attack). Assuming that brute force search is used in generating the other address in packet, whenever a spoofing packet (s,d) is detected, the most effective rule for the spoofing attack is the rule covering the largest flow space to deny, containing (s,d).

We can build a two-dimension bitmap for the flow space containing all the possible flows in the network, with each bit representing the finest granularity flow as Fig. 6. Mark each flow to deny by 1, and others by 0. Then for a spoofing packet, first find the corresponding bit on the map, and search the largest rectangle that covers the bit and can be represented as a single ACL rule.

The problem is a typical dynamic programming problem. Use (i,j) to denote the prefix length pair of aggregated flow space. In each search step, if only the bigger rectangle is full of 1, we try to reduce i or j by 1. Apparently, whenever the sum of i and j is minimum, the covered space is largest. The algorithm is described in Table 1.

Due to the constraint that both source and prefix spaces can be aggregated to one prefix, the search steps are generally quite short (no more than $32 * 32$ for IPv4, and $128 * 128$ for IPv6). The intermediate result of all 1 area is stored to avoid duplicate scan. The totally scanned bits are no larger than the number of bit 1. Similar mechanism can also be designed to generate 1-dimension filters, if semi-maximum filtering is used on routers.

If an attacker uses randomly generated address, the most effective rule is the rule which covers the largest triangle not covered by already configured rules. The problem is similar to the strategy under brute force attack. In the simulation, the random strategy is founded not as effective as brute force search because duplicate addresses will be generated and filtered by existing filters after a large number of trials.

Once filters are set, if a new spoofing packet from the same interface is found to be covered with existing filters, no new filter is needed to be configured.

A threshold is set to control the total number of filtering rules on routers. Once the threshold is reached, no new filters will be configured. Instead, an alarm is generated to notify the administrator. In such case, the best solution is to configure filters more close to attacking nodes.

Because usually spoofing based attacks are short but fierce, there is no need to perform filtering all the time. After a random time, the VASE server will tentatively remove the configured rules. If no spoofing is found for a period, the filtering rules can be removed permanently.

3. Evaluation and result

In this section, we evaluate VASE through comparing it with other direction based mechanisms in various scenarios based on simulation. We focus on two aspects of VASE: (1) The reduced data plane overhead; (2) The filtering capability. We also provide an analysis of the simulation result.

3.1. Measures

3.1.1. Overhead measures

We designed two measures to evaluate the data plane overhead:

- σ: the weighted average times per non-spoofing flow checked by routers. A flow is checked once if it passes an interface with filtering rule deployed. This measure is used to evaluate the extra processing cost of a mechanism on the non-spoofing packet forwarding. The time complexity of TCAM based ACL check is O(1). Because direction based mechanisms can always implement the filtering rules with TCAM based ACL, it is reasonable to use the check times as the process cost. Use k to denote the count of checking on the corresponding flow. Denote the relative weight of flow f with $\omega(f)$. In the simulation, $\omega(f)$ is determined by the relative flow space, i.e., the number of finest granularity flows covered by f. $\overline{SF}$ is used to denote the set of non-spoofing flows. Formally,

$$\sigma = \sum_{f \in \overline{SF}} k\omega(f) \Big/ \sum_{f \in \overline{SF}} \omega(f)$$

- τ: the ratio of deployed rules to all the rules on each interface. The measure is used to evaluate the ratio of saved entries on routers. Note that we do not use the amount of deployed rules, because it is an unfair

measure. Without considering false positive, the filtering capability of a direction based mechanism is determined by the amount of rules generated. The more rules generated by a mechanism, the more spoofing traffic can be filtered. For example, for any interface, a perfect direction based mechanism will generate all the rules corresponding to each flow that can/cannot pass the interface, but a simple mechanism may only generate a rule that filters traffic from an unassigned prefix. Though a mechanism may only have a small number of rules, it cannot filter as effective as another mechanism with a lot of rules. Thus, we use the deployed ratio as the measure.Denote the set of all the rules by *Rule*, and the set of deployed rules by $Rule_{deployed}$. Then,

$$\tau = |Rule_{deployed}|/|Rule|$$

3.1.2. Filtering capability measures

We evaluate the proposed filter generation mechanism using two measures:

- γ: the ratio that filtered legitimate traffic to all legitimate traffic whenever there are flows between all node pairs. This is the measure of false positive. Formally,

$$\gamma = \sum_{V(f)=false, f\in\overline{SF}} \omega(f) \Big/ \sum_{f\in\overline{SF}} \omega(f)$$

- θ: the ratio that filtered spoofing traffic to all spoofing traffic whenever all the nodes are attackers. This is the measure of false negative. Formally,

$$\theta = \sum_{V(f)=false, f\in SF} \omega(f) \Big/ \sum_{f\in SF} \omega(f)$$

3.2. Benchmarks

We compare VASE with other filtering mechanisms in practice: Ingress Filtering (IF) and uRPF. To avoid ambiguity, Ingress Filtering is defined to be a mechanism filtering traffic from stub network of deployed routers, and uRPF is defined to be a mechanism which filters traffic on all the interfaces of a deployed router based on forwarding table.

For a mechanism with static rule configuration, σ is determined by the deployment pattern of the mechanism. A self-discipline mechanism only checks the traffic at the first hop, but a protective mechanism may check traffic at the intermediate routers. In the evaluation of σ, Ingress Filtering represents any self-discipline mechanism, and uRPF represents any protective mechanism deployed on data path of traffic.

3.3. Simulation scenarios

We consider the following scenario parameters that may affect the result of the simulation:

3.3.1. Topology

Topology is one of the most crucial elements that affect the data path of a flow. For a direction based IP spoofing filtering mechanism, the pattern of the topology determines its effectiveness [14]. The topology pattern also affects the agility feature of VASE.

Considering the impact of topology, we evaluate VASE in various types of topology as listed in Table 2: real topology, inferred topology and synthetic topology.

We choose the topology of the campus network of Tsinghua University as the real topology, named by THU. There are 24 routers in the network, and 33 links between these routers. The backbone of THU is illustrated in Fig. 14, with some routers and links removed.

One inferred topology RF1755 from Rocketfuel [24], which is a router-level ISP topology inference engine, is used in the simulation. We remove the external links to the outside of the network, and remove the node unattached.

We use the Waxman model [25] to generate a random topology. Waxman- $N-\alpha-\beta$ denotes a topology with N nodes and parameters α and β. The interconnect probability between two nodes u and v is $\beta e^{-d(u,v)/\alpha L}$. We also used the HOT model proposed in [26], which is considered a better model of the real network than Waxman model.

3.3.2. Routing

We use the Shortest Path First algorithm to generate forwarding path on each topology. Link metrics are set to be constant. Because there can be multiple paths with the same cost between any node pair, we only choose one of the paths in the calculation. Thus, there can be asymmetric in routing as illustrated in Fig. 7. The more diversity the routing is, the more probable the asymmetric exists.

3.3.3. Deployment strategy

For any direction based filtering mechanism, deployment strategy is critical to the filtering capability. Because deploying the minimal Vertex Cover (VC) set of a network is found to be the most effective strategy [27], whenever choosing deployment node, we first choose nodes from

Table 2
Set of topologies.

Topology type	Topology name	Nodes	Links
Real	THU	24	33
Inferred	RF1755	300	548
Synthetic	Waxman-200-0.2-0.2	200	934
	HOT	200	224

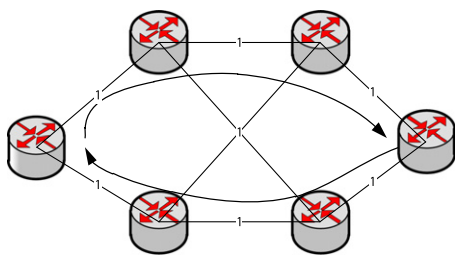


Fig. 7. Asymmetric routing resulted from multiple equivalent paths.

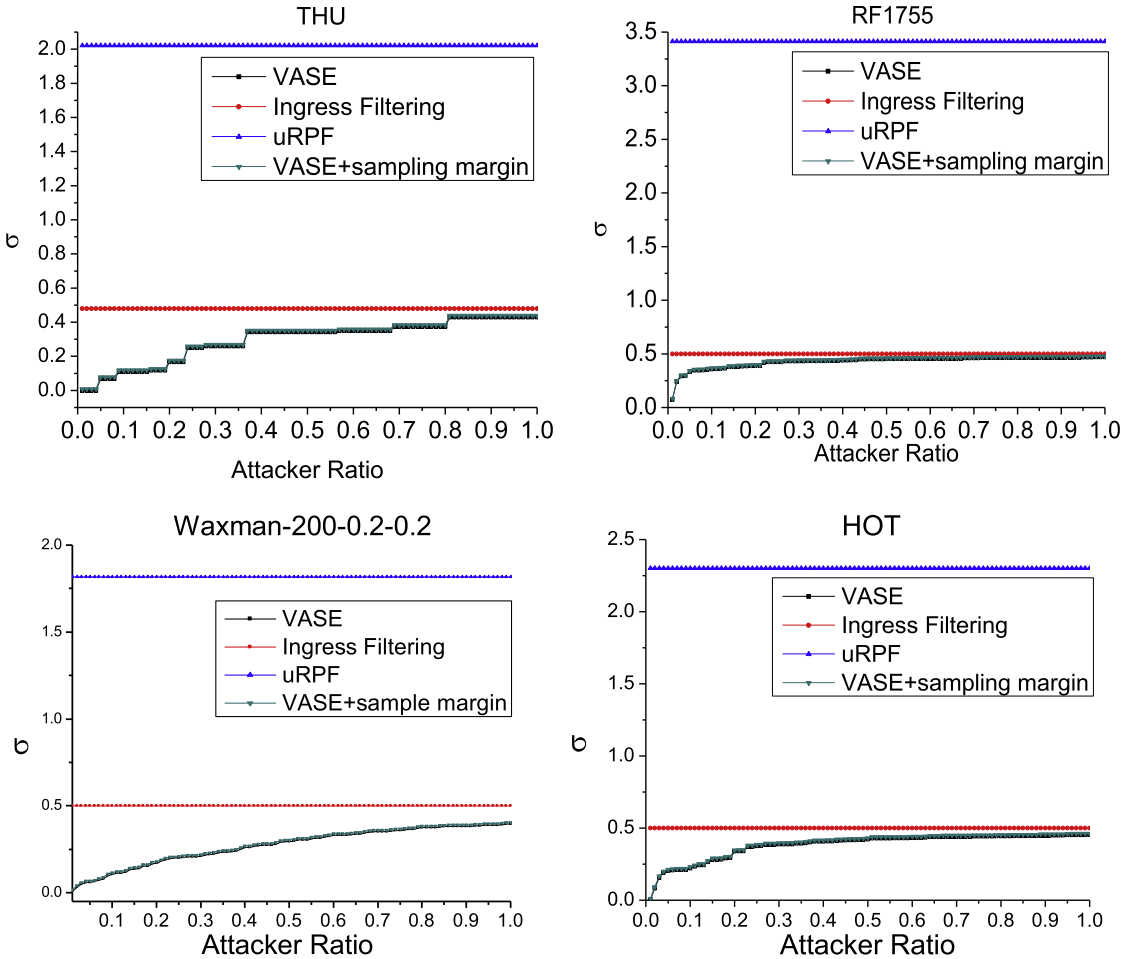


Fig. 8. σ in THU, RF1755, Waxman-200-0.2-0.2 and HOT with deployment ratio of 50%.

the minimal VC set. After that, the other nodes are chosen randomly. Evaluating the mechanisms using VC set represents the best performance of mechanisms. Mechanisms are deployed on the same selected nodes.

3.3.4. The observer coverage

In the case that VASE generates filtering rules based on flow path calculation, the spoofing filtering ability of VASE is affected by the coverage of router whose MIBs are collected. We consider two extreme situations: (1) MIBs are only collected from the nodes to deploy; (2) MIBs are collected from all the nodes in the network. The former is denoted by VE-L, and the latter is denoted by VE-U.

3.3.5. Traffic pattern

In the simulation, legitimate flows are launched between each node pair constantly. The relative volume of flow is determined by the relative flow space. For example, a (/23,/23) flow has four times volume of a (/24,/24) flow.

The pattern of attacking traffic is specified in each scenario.

3.4. Evaluation results

3.4.1. Data plane overhead

Fig. 8. shows the increase of σ with the increase of attacker ratio, with deployment ratio of 50%. In the simulation, half of the nodes are chosen to perform filtering. With each attacker ratio, the corresponding proportion of nodes is chosen to be attacker and launch spoofing traffic with random source to a random chosen victim.

Note that σ is the measure of additional process cost on non-spoofing traffic. It can be found that VASE can avoid packet filtering if there is no spoofing at all (attacker ratio = 0). Even when there is spoofing in the network, VASE only configures the necessary filtering rules; thus, the process cost is still lower than Ingress Filtering and uRPF. Due to the perimeter, VASE checks a flow once at the most. In case of 50% deployment ratio, the upper bound of σ (VASE) is 0.5. This cost is equal to (Ingress Filtering). A flow can be checked by uRPF for multiple times on the path. Thus, σ (uRPF) can be more than 1.

VASE has sampling cost. The sampling margin is also plotted in Fig. 8. Generally, the sampling rate is quite low

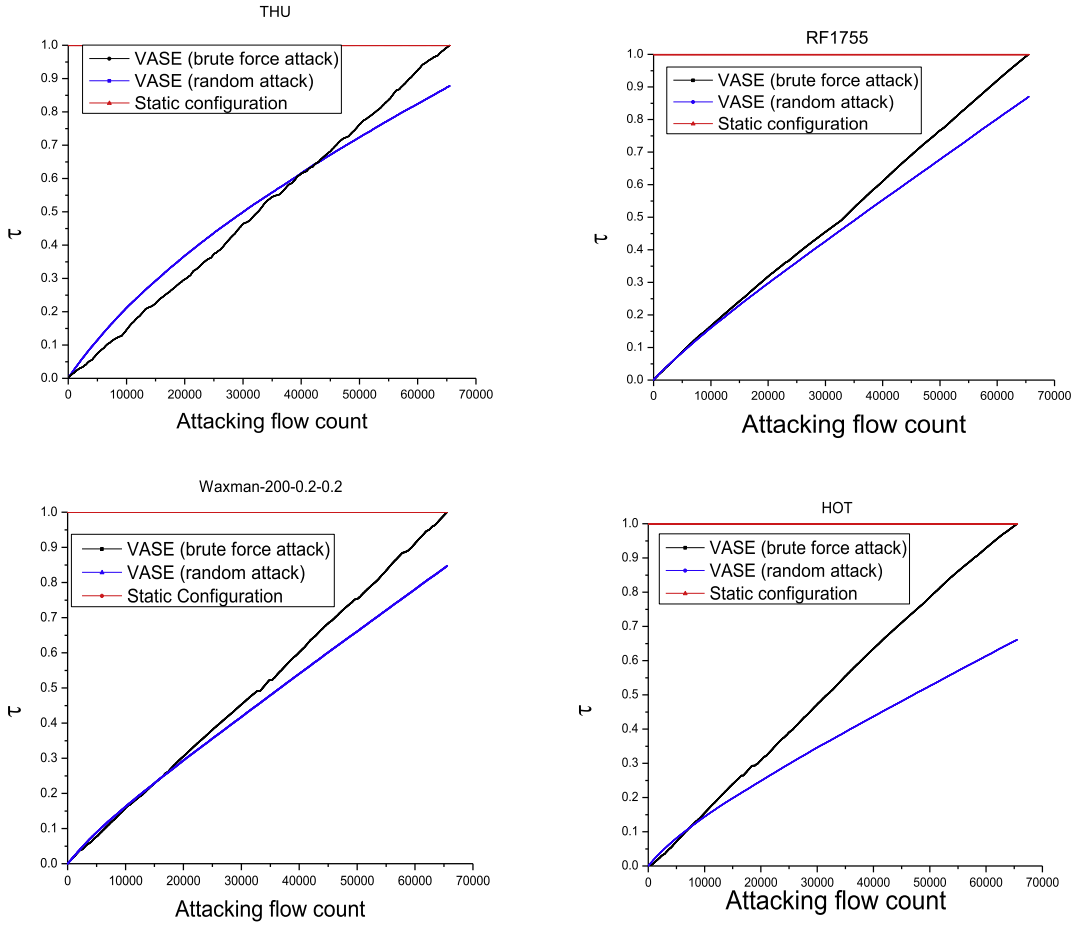


Fig. 9. τ of THU, RF1755, Waxman-200-0.2-0.2 and HOT.

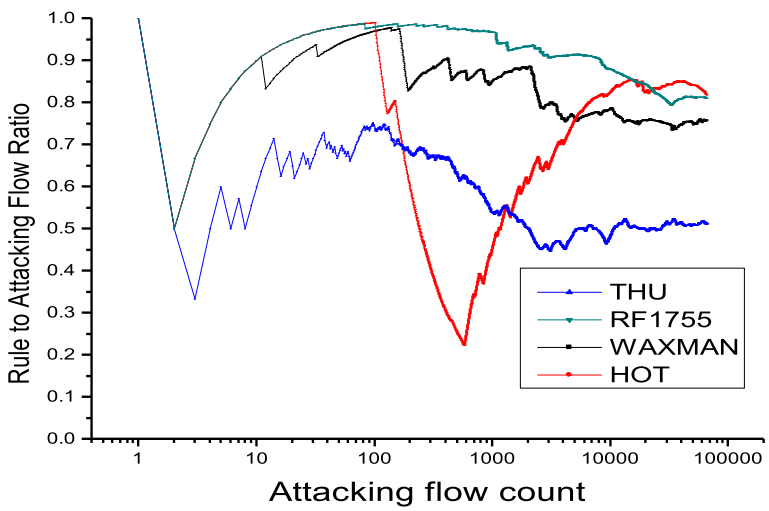


Fig. 10. The configured rule number to attacking flow count ratio.

and the average process cost on each flow is only slightly increased. The total cost of the data plane is still far below the cost of the other mechanisms.

Through a comparison of VASE in different topologies, the impact of topology on VASE can be got. Apparently, *power-law goes against agility*. For example, in RF1755

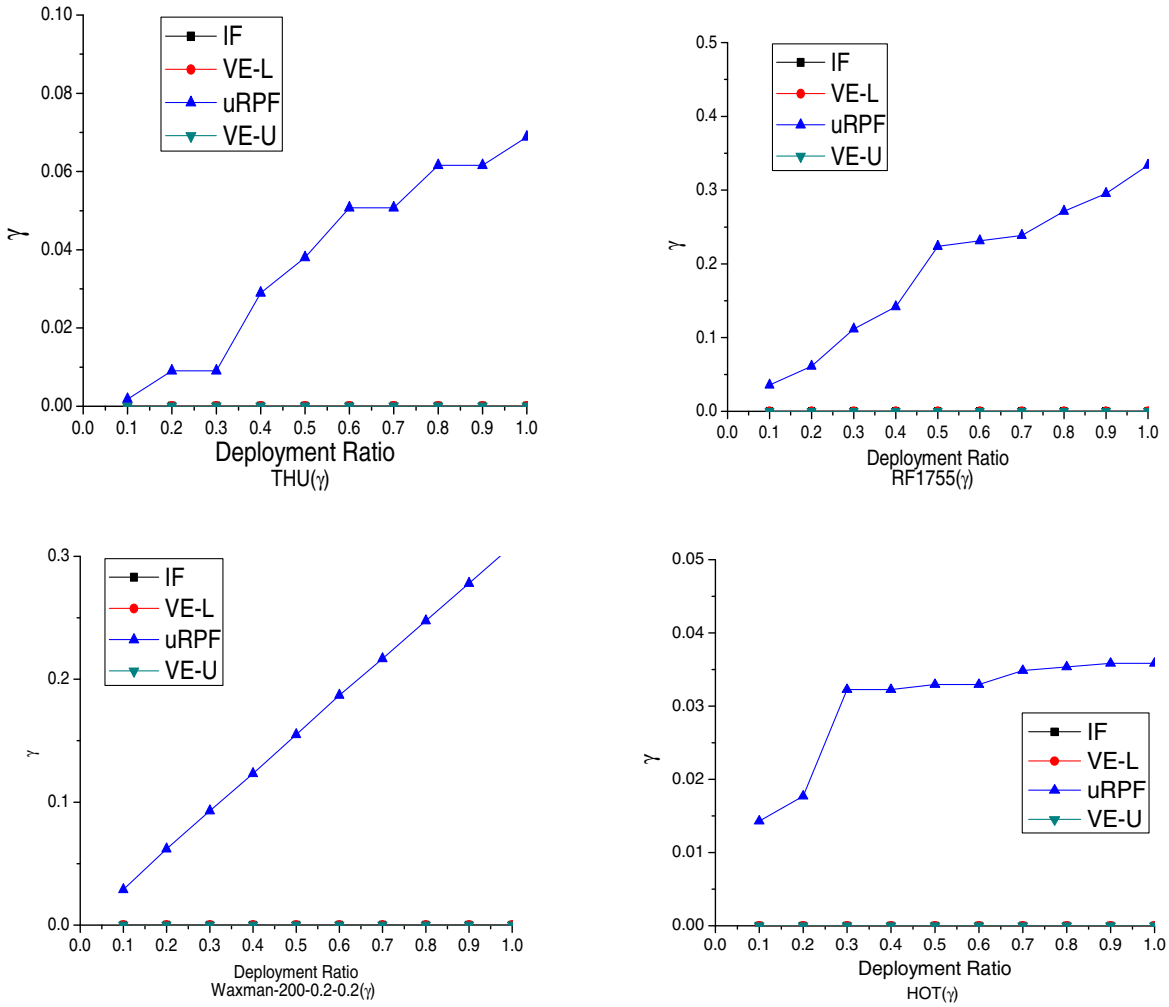


Fig. 11. γ of THU, RF1755, Waxman-200-0.2-0.2 and HOT.

which follows power-law, σ (VASE) approaches (Ingress Filtering) fast; in the random topology Waxman-200-0.2-0.2, the increase of σ (VASE) is much slower. The reason is explicit: a filter on a node will introduce cost on flows passing by; a node of high rank can play as the intermediary of a large number of paths in a network following power-law.

Fig. 9. shows the increase of τ to the number of forged addresses of attackers. The result is got through calculating the ratio of configured filters in scenarios that an attacker uses brute force search generated source and randomly generated source. In the simulation, 65,535 prefixes are distributed randomly to nodes in the network. In each search step, the selected attacker chooses a forged address to launch traffic to a random victim.

The result shows a static mechanism configures all the rules all the time, but the configured rule ratio of VASE is proportional to the spoofing space of an attacker. This means VASE can avoid occupying unnecessary resource to filter out spoofing flows. Only the rules currently need are configured in VASE. Also, we find a random spoofing strategy is not as effective as brute force search with the increase of attack times. The topology has impact on the increase of τ under random spoofing strategy. For example, in HOT, the increase of τ is quite slow. It is due to HOT is a tree-like topology whose FlowTable can be well aggregated. Thus, a small number of rules can cover most of the spoofing space.

Fig. 10. shows the ratio of configured rules to the number of attacking flows under brute force strategy. The ratio is obviously less than 1. This result shows the Maximum Rectangle algorithm is effective to find the suitable rule to cut spoofing traffic. With the Maximum Rectangle algorithm, there is no need to configure a rule for each spoofing flow. A spoofing flow can be covered by a deployed rule. This result implies an attacker must generate more spoofing flows than configured rules by VASE. Also, the topology has impact on the ratio because it determines how well the FlowTable can be aggregated.

3.4.2. Filtering capability

3.4.2.1. False positive. Fig. 11. shows the false positive of mechanisms in each topology. Unlike other mechanisms, uRPF is found of more or less false positive in each

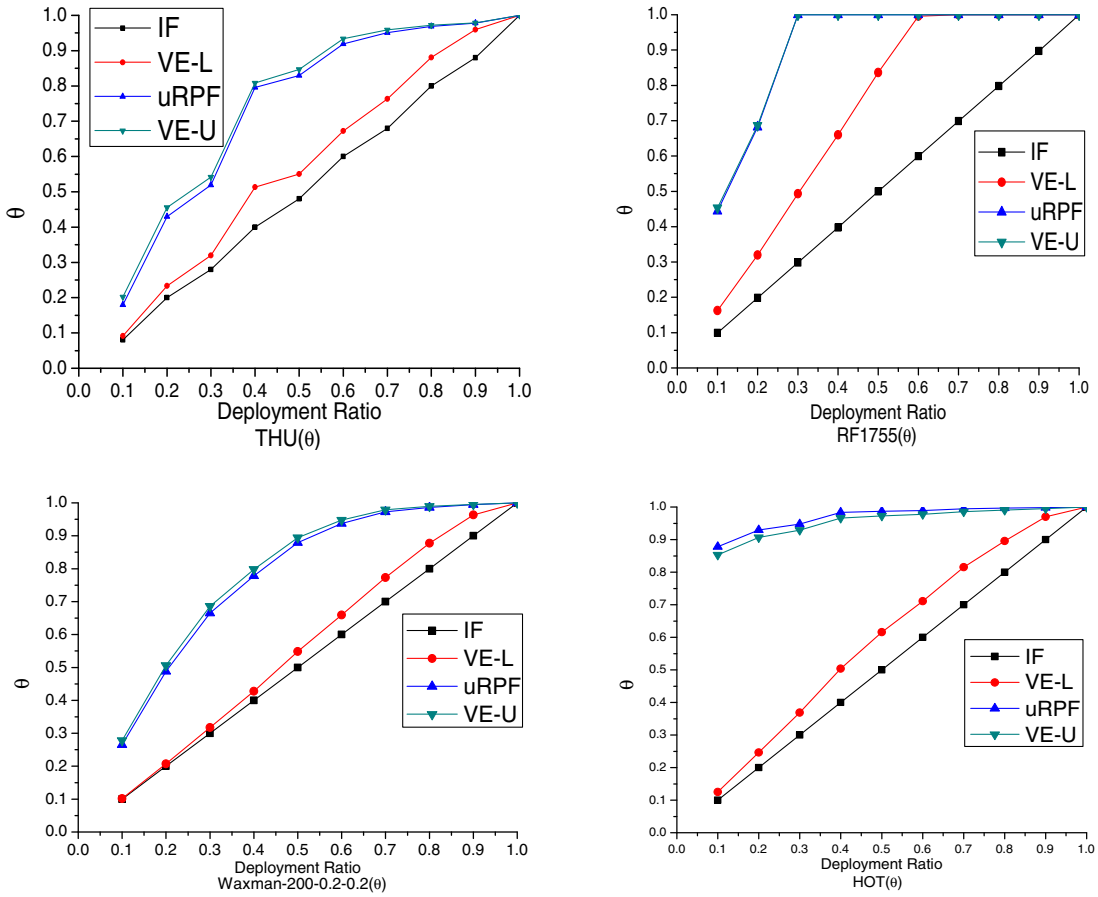


Fig. 12. θ of THU, RF1755, Waxman-200-0.2-0.2 and HOT.

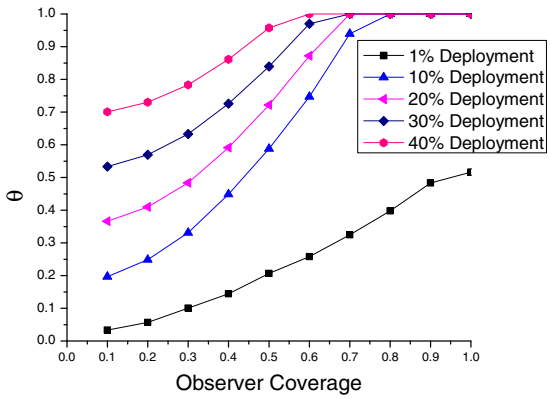


Fig. 13. θ of RF1755 with the increase of observer coverage.

topology. This result implies uRPF is not a favorable protective mechanism for current networks. In fact, based on the statistic from MIT Spoofer project [9], most of the spoofing traffic is filtered by the first router on the path. This implies in realty, uRPF is used as a self-discipline mechanism, but not a protective mechanism, because such deployment cannot protect the network from spoofing launched by other networks. If uRPF is performed on a port not directly attached by stub networks, it may discard legitimate traffic due to asymmetric routing.

3.4.2.2. False negative. Fig. 12. shows the filtered spoofing traffic ratio in each topology. VASE-U and uRPF are found to be of approximate filtering capability. However, considering the false positive of uRPF, VASE-U is more preferable. VASE-L has some advantage than Ingress Filtering in all topologies. In RF1755, VASE-L has an obvious advantage against Ingress Filtering, resulted from the typical power-law structure of RF1755. Considering the power-law feature is becoming more and more notable in current networks, even only cooperating a small fraction of nodes, the increased filtering capability can be obvious.

To illustrate the effect of route information collection coverage to filtering capability, the filtering capability increase against the observer (the router to collect route information) coverage ratio increase is plotted in Fig. 13. The result shows the benefit of the global view on VASE server to the filtering capability. With better knowledge of the network routing, the spoof filtering can achieve better capability. Even when the number of nodes chosen to perform filtering is small (1%), with a global view of the network, 50% spoofing traffic can be filtered out.

4. Implementation and deployment

A prototype of the VASE has been implemented. The source code of this prototype can be fetched from http://netarchlab.tsinghua.edu.cn/~yaog/vase/src/.

We use sFlow [28] and NetFlow [29] on routers to sample traffic. The export formats of different sampling mechanisms are not identical. A tetrad (*Router, Interface, Source, Destination*) is used to represent the sampling result. VASE server collects sampled packets and extracts the tetrads from the packets. The tetrads are inserted into a buffer which is used to handle burst of samples. The set structure is used to keep each tetrad unique to avoid checking on duplicate tetrads.

VASE server collects routing table MIB and interface table MIB on routers through the SNMP protocol, and generates filtering rules to validate the samples. Because no standard ACL MIB exists, to configure the Filtering Table on each VASE node automatically, we choose to download the rules using script through SSH. Thus, because the user interface syntaxes of different vendors are different, the script is generated on predefined template.

The implementation has been deployed in IPv6 campus network of Tsinghua University, as illustrated in Fig. 14. Two perimeters are formed after the deployment. The routers outside the VASE are not under the control of network operation group, and the forwarding choices of the routers are unpredictable; thus uRPF is unsuitable to be deployed due to the potential false positive. Because NetFlow has been enabled on the VASE interface already for management purposes, the deployment of the VASE brings no additional overhead on the data plane. In spite of the trivial deployment cost, if all the hosts in the campus network attend in attacks against nodes in the network or out of the network, 89% spoofing flows can be filtered in average according to the simulation using real routing table. Indeed, VASE has been deployed in the IPv6 campus network of 100 universities in China.

We got some performance data from the implementation and deployment in Tsinghua University. The average sample collection rate on VASE server is around 1200 pps. Each sample check procedure can process around 300 packets each second (note that there are generally more than one flows in each NetFlow message); thus we run 8 procedures in parallel to handle a possible burst. The information collection and filtering rule generation cost 2.3 s in average, and the highest observed value is 5.1 s till the submission. The average re-active time, including the sample check, spoofing confirmation and filter configuration, is 2.8 s in the experiments. This value shows VASE can react fast enough.

Till the submission of this paper, the deployed VASE mechanism has worked for 10 months. Though currently the spoofing DoS attacks in IPv6 world are rare (on the other hand, we have deployed SAVI solutions [30] on most of the switches), thousands of spoofing events caused by wrong configuration of a 6to4 tunnel are detected and verified in the period. In the captured events, a host played

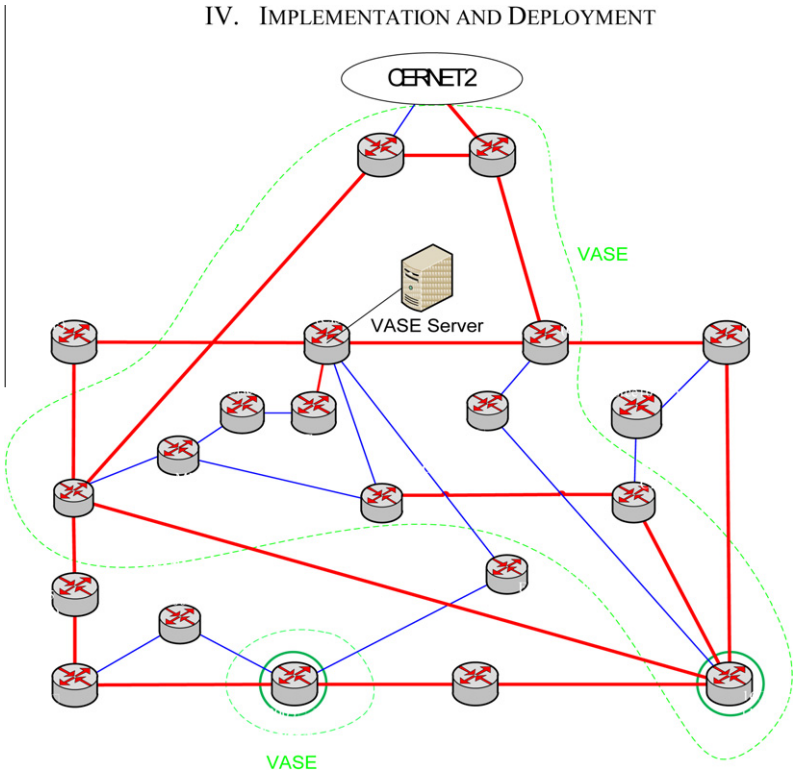


Fig. 14. The deployment of VASE in the IPv6 campus network of Tsinghua University.

as an exit of an unauthorized 6to4 tunnel. A ratio of IPv6 traffic between other campuses, which should not pass the campus network of Tsinghua University, was attracted by the tunnel. As a result, it appeared the host was sending traffic with forged sources. Besides the events caused by the tunnel, only one filter was deployed on an edge router for hundreds of times to cut a spoofing flow with source set to an address inside the perimeter, which seemed to be caused by a host with wrongly configured IP address. Indeed, in most of the time, no filtering is performed at all. This result proves the deployment of VASE can significantly reduce the unnecessary cost of IP spoofing filtering in peace time.

5. Related work and background

5.1. IP spoofing prevention mechanisms

As a long recognized security threat, IP spoofing attacks have been prevented by Ingress Filtering and IPsec [31] for a number of years. With Ingress Filtering, if a packet is not with the source address of the network connected to its incoming interface, it is getting filtered. The filtering is usually performed by manually configured ACL or uRPF check. There have been numerous Best Current Practices (BCPs) (BCP38, BCP84) and IETF RFCs about Ingress Filtering. Ingress Filtering is also a well-supported function on commodity routers. Standardization and universally support make Ingress Filtering widely deployed. Ingress Filtering is generally enabled at edge devices. In most cases, it relies on the uRPF feature to build the correct filtering table. On core routers, because there is no explicit binding between prefix and interface, only traffic with bogus prefixes is filtered to prevent spoofing.

IPsec is employed in many scenarios supplying the ability of authenticating the origin of the packet. The major problem of IPsec is that global distribution of the key is still a challenge. Besides, IPsec is vulnerable to DDoS attack because the attacking traffic has aggregated at the victim host, and the computation cost of authentication is generally not low enough.

There also have been a number of research works, which can be classified into two categories.

1: End-to-end authentication. Such mechanisms validate the source address at the other end of communication (at different granularity). [18] proposed a mechanism which inserts AS tag into data packet and check the binding between address prefix and AS tag at the destination AS. This mechanism requires the association of ASs and update of routing devices. HCF [19] checks the validation of source prefix based on the binding between prefix and hop count value which is inferred from remaining TTL value. HCF is light-weighted, and of no deployment requirement, but it has significant false negative because the forged network and the attacker can have non-trivial probability to have the same hops away from the destination, and it can be bypassed by smart attackers who modify the initial TTL value. Also, it can result in false positive when the hop count value is inconsistent, e.g., in case of route change.

2: Filtering-on-path. Such mechanisms filter packets on the path, generally before the packet aggregated at victim network. [14] proposed a direction based filtering mechanism, which filters packets based on the binding between prefix/flow and interface. However, the article does not propose a universal mechanism to build the filtering table. [17] proposed a mechanism that checks cryptographic signature hop-by-hop at AS granularity. It can filter spoofing traffic before they aggregate, and the filtering can be performed at AS granularity, which is better than general filtering-on-path mechanisms. However, the cost is also heavy: it requires the cooperation of ISPs, update of routing device, the update of client and modification of the packet. [15] proposed a new protocol on routing devices, which announce the local route selection choice and help others build the mapping table from incoming prefix to the interface. [16] builds inter-domain filtering rules based on the valley-free feature of inter-domain routing and the BGP announcement filtering rules.

5.2. Related functions on commodity routers

In order to avoid updating routers to adopt this mechanism, the following functions widely supported on routers can be exploited:

1. *XFlow:* XFlow is used to represent sFlow [28] or NetFlow [29] or other mechanisms on the router with sampling function. Generally such mechanisms are commonly configured to monitor traffic pattern.
2. *ACL:* Whenever needed, ACL rules on routers are configured to enforce filtering. ACL performed by the hardware can achieve line-speed; thus the capacity of routers will not be affected.
3. *SNMP and MIBs:* SNMP can be used to fetch the routing table and interface MIB on routers. The MIB content is used to generate correct filtering rules. For management purposes, SNMP is usually enabled in networks.

6. Discussions, conclusions and future work

6.1. Discussions

6.1.1. Discussions on comparison with uRPF

Compared with uRPF, which is the most common IP spoofing filtering mechanism, VASE requires deploying an additional server. It seems an additional overhead, but generally there has been a server deployed in the network helping the network management. For example, a server may be used to collect MIBs from routers with SNMP. The server can be used as the VASE server. Besides, VASE can reduce data plane cost if only there is no permanent spoofing in the network.

Though uRPF is universally used, it has its own weaknesses. Considering the false positive caused by asymmetric routing, uRPF is generally enabled on the edge

routers. On the core routers, uRPF is not enabled, but non-directional ACL is used instead [32]. Such a deployment is actually self-discipline: it can filter spoofing from local edge networks, and cannot protect the network from spoofing traffic launched by other domains. However, VASE does not introduce false positive in case of asymmetric routing. Thus, VASE can enable IP spoofing filtering on the core routers, and protect routers inside the VASE perimeter from spoofing originated outside. Such a feature is essential to protect core infrastructures. Important servers in a network are generally protected by access control rules which only allow access from specified IP addresses. The access control rules cannot prevent the servers from spoofing DoS traffic with source set to the allowed IP addresses. However, VASE can filter out such spoofing traffic and provide better protection.

Another weakness of uRPF is that it discards traffic silently. It is difficult for the administrator to diagnose the problem when legitimate traffic is discarded by uRPF, and it is difficult for the administrator to notice an attack and take further actions. VASE can identify each spoofing explicitly. This is the reason why we can detect the address misuses in campus network of Tsinghua University.

6.1.2. Security considerations on VASE

It is necessary to ensure the safety of the VASE server and the channels between VASE server and routers. With the VASE perimeter, a straightforward solution can be given: place the VASE server inside the perimeter. In this way, the VASE server can be protected from spoofing attacks. Through setting the VASE server to be accessible only from the management addresses of routers, the VASE server can be prevented from attack from malicious nodes, even with spoofing source IP address. Through setting the management addresses of routers to be accessible only by the IP address of VASE server, the channels between the VASE server and the routers can be protected. In the campus network of Tsinghua University, we just configure the IP address of the VASE server to an address belongs to the management network which owns a specified prefix, and prevent the management network from access from other prefixes.

Note that if there is more than one perimeter, the channels between the VASE server and the routers inside the different perimeters can be accessible from malicious nodes through spoofing. However, the channels can still be protected by the authentication mechanisms of SNMP and SSH, which are commonly enabled in network management practice. At least, the deployment of VASE does not introduce more risk than existing protocols.

6.2. Conclusions

In this article, we proposed the mechanism to build a Virtual Anti-Spoofing Edge that filters spoofing traffic with agility, i.e., with flexible cost and resource requirement. Sampling, passive filtering generation mechanism and adaptive filter configuration significantly reduce the cost when no spoofing or little spoofing happens. VASE is compared with other mechanisms based on simulation, and obvious advantage is found. We have implemented a prototype of VASE and deploy it in the running network to prove this mechanism.

The primary design goal of VASE is practicability. Thus, no modification on routers is made. However, because of this restraint, VASE is not a state-of-art solution, e.g., the sampling cost, the outer-sync problem with filter generation, and the filter deployment complexity of configuration through script. In the future, we plan to implement a VASE that can cooperate with OpenFlow [21], and better resolve the existing problems.

6.3. Future work: OpenFlow based solution

OpenFlow protocol can benefit VASE in sampling, filtering generation and filter configuration:

6.3.1. Sampling

The sampling rate is designed to capture most of the significant attack. However, small attacks may not be found by the mechanism. Ideally, performing sampling on each flow only once is the most effective sampling mechanism to detect spoofing. In this situation, no sampling is wasted, and all the spoofing flow can be sampled. In OpenFlow, a packet-in event will be sent to the controller on each packet not matching any entry. This function can be used to replace the sampling mechanism. Flow entry associated with the event can be set to avoid future collection on the same flow. In this way, each flow can be sampled once and only once.

6.3.2. Filter generation

The OpenFlow controller can tightly track the state of routers through collecting the asynchronous messages from routers. Moreover, the controller can even take control of the flow tables on routers. Thus, filtering rules can be generated more consistent with instant routing state with OpenFlow.

6.3.3. Filter configuration

The protocol of OpenFlow provides a uniform interface for the controller to modify the flow table on routers. Thus, the differences in router UI syntax can be eliminated. Filters can be easily configured on any router that supports the OpenFlow protocol.

We have implemented a similar mechanism based on OpenFlow/NOX for access networks [33], as a complement of SAVI [30]. However, the intra-domain network is different from the access network. The topology of the access network is simple, and a controller can be used to determine all the data paths. However, an intra-domain network is more complex. It is of considerable cost to take complete control of all the data paths in an intra-domain network, and generally it is not necessary. Thus, we proposed OpenRouter [34], which supports OpenFlow protocol but routes are generated mostly based on routing protocols. We are implementing a VASE with OpenRouter.

Acknowledgements

This work is supported by National Science Foundation of China under Grant 61073172, Program for the New

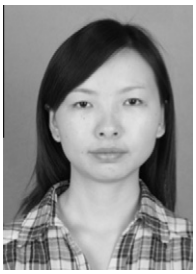
Century Excellent Talents in University, Specialized Research Fund for the Doctoral Program of Higher Education of China under Grant 20090002110026, and National Basic Research Program ("973" Program) of China under Grant 2009CB320501. We are grateful to anonymous reviewers for the comments that helped us improve the manuscript.

References

[1] S.M. Bellovin, Security problems in the TCP/IP protocol suite, SIGCOMM Computer Communication Review 19 (2) (1989) 32–48.
[2] CERT, TCP SYN Flooding and IP Spoofing Attacks, 1996. <http://www.cert.org/advisories/CA-1996-21.html>.
[3] CERT, Smurf IP Denial-of-Service Attacks, 1998. <http://www.cert.org/advisories/CA-1998-01.html>.
[4] R. Vaughn, G. Evron, DNS Amplification Attacks, 2006. <http://www.isotf.org/news/DNS-Amplification-Attacks.pdf>.
[5] ICANN, SSAC Advisory SAC008 DNS Distributed Denial of Service (DDoS) Attacks, 2006.
[6] D. Moore et al., Inferring Internet denial-of-service activity, ACM Transactions on Computer Systems 24 (2) (2006) 115–139.
[7] C. Labovitz, Bots, DDoS and Ground Truth, 2011. <http://www.nanog.org/meetings/nanog50/presentations/Tuesday/NANOG50.Talk58.groundtruth.pdf>.
[8] P. Ferguson, D. Senie, Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing(RFC2827), 2000. http://tools.ietf.org/rfc/rfc2827.txt.
[9] R. Beverly et al., Understanding the efficacy of deployed internet source address validation filtering. IMC'09: Proceedings of the 2009 Acm Sigcomm Internet Measurement Conference, 2009, pp. 356–369.
[10] Arbor, Worldwide Infrastructure Security Report, 2010. <http://www.arbornetworks.com/dmdocuments/ISR2010_EN.pdf>.
[11] F. Baker, P. Savola, Ingress Filtering for Multihomed Networks (RFC3704), 2004. <http://www.ietf.org/rfc/rfc3704.txt>.
[12] D. Rajnovic, Black Hole Routers, 2002. <http://www.terena.org/activities/tf-csirt/meeting7/rajnovic-black-hole-routers.pdf>.
[13] D. Meyer, L. Zhang, K. Fall, Report from the IAB Workshop on Routing and Addressing, 2007. <http://tools.ietf.org/html/draft-iab-raws-report-01>.
[14] K. Park, H. Lee, On the effectiveness of route-based packet filtering for distributed DoS attack prevention in power-law Internets, Computer Communication Review 31 (4) (2001) 15–26.
[15] J. Li et al., SAVE: source address validity enforcement protocol, in: Proceedings of the IEEE Infocom 2002: The Conference on Computer Communications, vols. 1–3, 2002, pp. 1557–1566.
[16] Z.H. Duan, X. Yuan, J. Chandrashekar, Constructing inter-domain packet filters to control IP spoofing based on BGP updates, in: Proceedings of the IEEE Infocom 2006: 25th IEEE International Conference on Computer Communications, vols. 1–7, 2006, pp. 510–521.
[17] X. Liu et al., Passport: secure and adoptable source authentication, in: Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation 2008, USENIX Association: San Francisco, California, 2008, pp. 365–378.
[18] A. Bremler-Barr, H. Levy, Spoofing prevention method, in: Proceedings of the IEEE Infocom 2005: The Conference on Computer Communications, vols. 1–4, 2005, pp. 536–547.
[19] H.N. Wang, C. Jin, K.G. Shin, Defense against spoofed IP traffic using hop-count filtering, IEEE–ACM Transactions on Networking 15 (1) (2007) 40–53.
[20] A. Greenberg et al., A clean slate 4D approach to network control and management, Computer Communication Review 35 (5) (2005) 41–+.
[21] N. McKeown et al., OpenFlow: enabling innovation in campus networks, Computer Communication Review 38 (2) (2008) 69–74.
[22] M. Casado et al., Ethane: taking control of the enterprise, Computer Communication Review 37 (4) (2007) 1–12.
[23] K. Levenberg, A method for the solution of certain non-linear problems in least squares, Quarterly of Applied Mathematics 2 (1944) 164–168.
[24] N. Spring et al., Measuring ISP topologies with rocketfuel, IEEE–ACM Transactions on Networking 12 (1) (2004) 2–16.
[25] B.M. Waxman, Routing of multipoint connections, IEEE Journal on Selected Areas in Communications 6 (9) (1988) 1617–1622.
[26] L. Li et al., A first-principles approach to understanding the Internet's router-level topology, Computer Communication Review 34 (4) (2004) 3–14.
[27] B. Armbruster, J.C. Smith, K. Park, A packet filter placement problem with application to defense against spoofed denial of service attacks, European Journal of Operational Research 176 (2) (2007) 1283–1292.
[28] P. Phaal, S. Panchen, N. McKee, InMon Corporation's sFlow: A Method for Monitoring Traffic in Switched and Routed Networks(RFC3176), 2001. <http://tools.ietf.org/rfc/rfc3176.txt>.
[29] CISCO, Introduction to Cisco IOS NetFlow – A Technical Overview, 2007. <http://www.cisco.com/en/US/prod/collateral/iosswrel/ps6537/ps6555/ps6601/prod_white_paper0900aecd80406232.html>.
[30] J. Wu, J. Bi, M. Bagnulo, F. Baker, C. Vogt, Source Address Validation Improvement Framework, 2011.
[31] S. Kent, R. Atkinson, IP Encapsulating Security Payload (ESP) (RFC2406), 1998.
[32] CISCO, Protecting Your Core: Infrastructure Protection Access Control Lists, 2008. <http://www.cisco.com/application/pdf/paws/43920/iacl.pdf>.
[33] Y. Guang, B. Jun, X. Peiyao, Source address validation solution with OpenFlow/NOX architecture, in: 19th IEEE International Conference on Network Protocols (ICNP), 2011.
[34] F. Tao, B. Jun, H. Hongyu. OpenRouter: OpenFlow extension and implementation based on a commercial router, in: 19th IEEE International Conference on Network Protocols (ICNP), 2011.

Guang Yao received the B.S. and Ph.D. degree in computer science from Tsinghua University. Currently he is a Post-Doctoral fellow of Tsinghua University.

Jun Bi received the B.S., M.S., and Ph.D. degree in computer science from Tsinghua University. Currently he is a full professor and director of Network Architecture & IPv6 Research Division, Network Research Center of Tsinghua University, Beijing, China.



Peiyao Xiao received the B.S. degree in computer science from Tsinghua University. Currently she is working toward a Master's Degree.